

Dossier d'Architecture

Projet DV2026 - EPHEC | Paul MALIOUKOV - PSR10219

Table des matières

Introduction	1
Structure	2
Architecture Backend	3
Architecture Frontend	5
Librairies, Outils et Frameworks Structurants	7
Documentation de l'API	11
Conclusion	11

Introduction

Ce document décrit l'architecture technique du projet *ProjetDV2026-PMALIOUKOV*, développé dans le cadre des cours *561/1 IN3-Projet développement Web* et *562/1 IN3-Projet de développement SGBD* à l'EPHEC (2026).

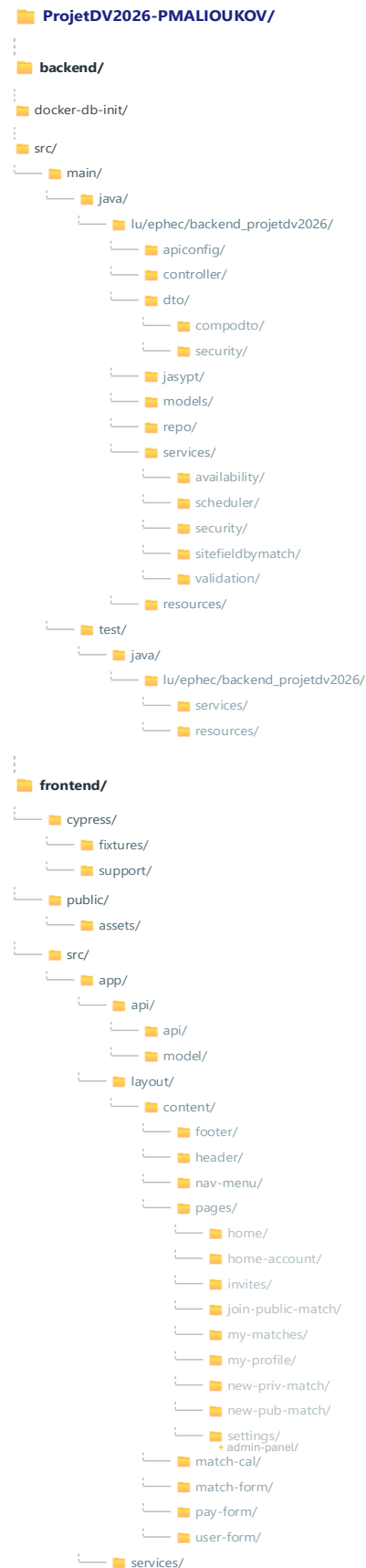
Le projet suit une architecture modulaire et séparée en couches pour assurer une maintenabilité optimale, une scalabilité et une séparation claire des responsabilités.

Repo Git : [ProjetDV2026-PMLKV](#)

Fichier d'exploitation du projet : [README.md](#)

Structure

Le projet adopte une architecture mono-repo organisée comme suit :



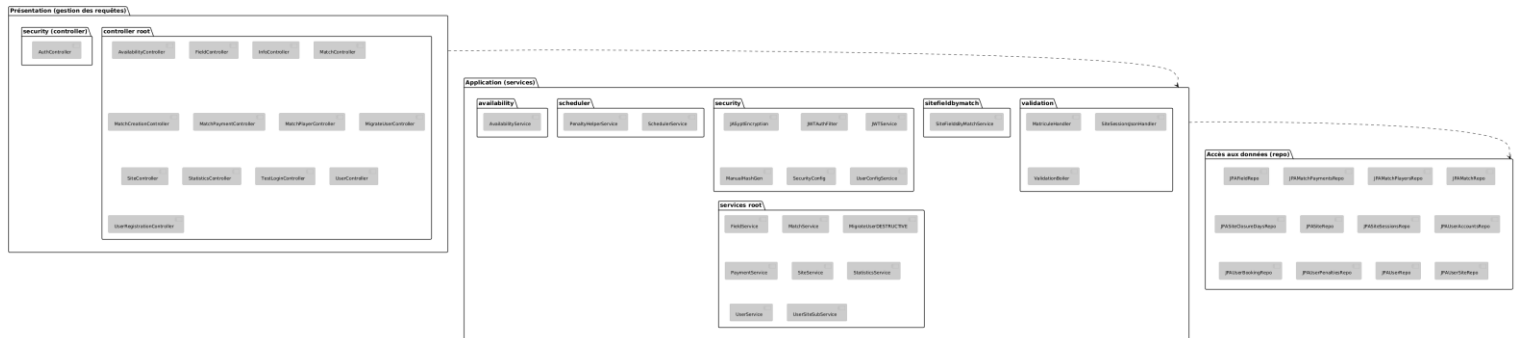
📁 = Directory

Project: ProjetDV2026-PMALIOUKOV/

Architecture Backend

L'architecture backend suit un modèle **3-tier (3 couches)** avec une séparation stricte des responsabilités.

Hiérarchie des Couches :



Le schéma hiérarchique peut être consulté plus en détails sur [PlantUML](#).

Couche Présentation (Couche inférieure)

- Rôle : Gestion des requêtes HTTP entrantes.
- Responsabilités :
 - Réception des requêtes HTTP (REST API).
 - Appel du service approprié.
 - Gestion des autorisations (appel service JWT).
 - Formatage des réponses (JSON et codes HTTP).
- Technologies :
 - Spring Boot
 - Spring Doc OpenAPI
 - Spring Web

Couche Application / Services (Couche intermédiaire)

- Rôle : Contient la logique métier et l'orchestration.
- Responsabilités :
 - Ordonnancement des opérations.
 - Validation des règles.

- Appel des repositories pour accéder aux données.
- Vérification des autorisations (JWT et Spring Security)
- Transformation des données (DTOs).
- Technologies :
 - Spring Boot
 - Spring Service
 - Spring Security
 - Spring Scheduler
 - JASypt
 - Jackson
- Tests :
 - JUnit
 - Javafaker

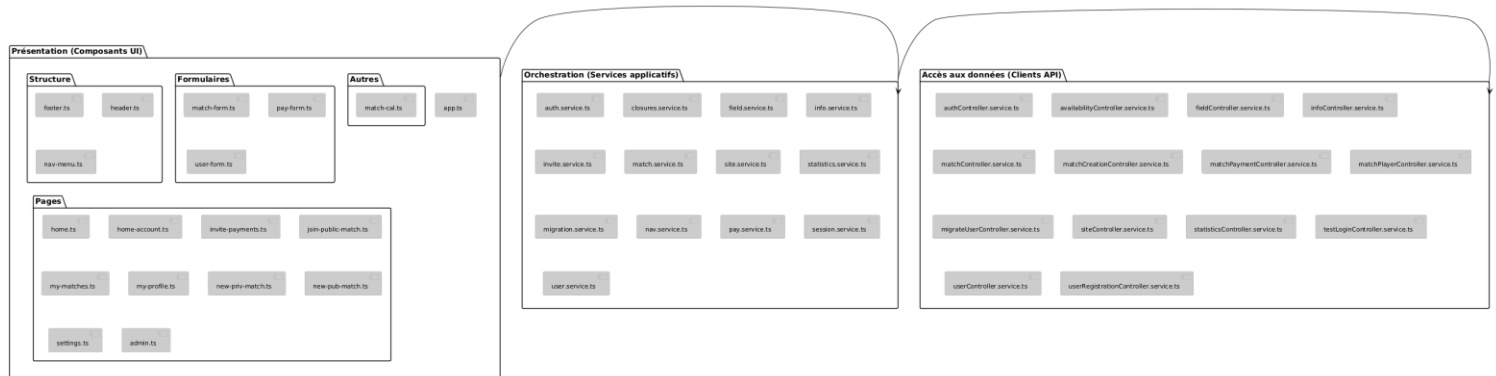
Couche Accès aux Données (*Couche supérieure*)

- Rôle : Interaction avec la base de données.
- Responsabilités :
 - Récupération des données.
 - Transformation des données (modèles).
 - Exécution des requêtes JPA/Hibernate.
- Technologies :
 - Spring Data JPA
 - Jdbc
 - Lombok
 - Hibernate
 - Microsoft SQL Server / Docker
- Tests :
 - H2 database

Architecture Frontend

L'architecture frontend suit un modèle **basé sur les composants** avec une séparation claire en **trois couches**.

Hiérarchie des Couches :



Le schéma hiérarchique peut être consulté plus en détails sur [PlantUML](#).

Couche Présentation (Composants)

- Rôle : Gestion de l'interface utilisateur et de l'expérience utilisateur.
- Responsabilités :
 - Affichage des données.
 - Gestion des interactions utilisateur (clics, formulaires, etc.).
 - Appel du service approprié.
 - Application des styles.
 - Logique de présentation (affichage conditionnel, animations).
- Technologies :
 - Angular
 - Tailwind CSS
 - Angular Material
 - RxJS
 - TypeScript
 - HTML5

- Tests :
 - Cypress
 - Karma-jasmine

Couche Service (*Orchestration*)

- Rôle : Coordination entre la couche de présentation et la couche de données.
- Responsabilités :
 - Appels aux API backend
 - Transformation des données
 - Gestion des erreurs
 - Logique métier côté client
- Technologies :
 - Angular Services
 - RxJS

Couche Accès aux Données (*Client HTTP*)

- Rôle : Communication avec le backend avec usage de Open API.
- Responsabilités :
 - Requêtes HTTP (GET, POST, PUT, DELETE).
 - Interception des requêtes/réponses (ajout de headers, gestion des erreurs globales).
 - Définition des modèles de données (interfaces TypeScript).
- Technologies :
 - Angular
 - Open API generator
 - HttpClient (Angular)

Librairies, Outils et Frameworks Structurants

Backend

Catégorie	Technologie	Version	Rôle
Framework	Spring Boot	4.0.3	Framework principal pour le développement backend.
Framework	Spring Web MVC	-	Fournit tout le nécessaire pour créer des applications web et des API REST (serveur web embarqué, gestion des requêtes HTTP, etc.).
Sécurité	Spring Security	-	Gestion de l'authentification et des autorisations.
Sécurité	JWT	0.12.6	JSON web token pour la gestion du token d'authentification et des autorisations.
Accès aux données	Spring Data JPA	-	Accès aux données avec JPA.
Accès aux données	JDBC	-	Couche de traduction pour l'accès aux données d'une base de données MS SQL.
ORM	Hibernate	-	Mapping Objet-Relationnel.
Validation	Hibernate Validator	-	Validation des données.
API Documentation	SpringDoc OpenAPI	2.5.0	Génération automatique de la documentation Swagger.
Utilitaire	Jackson	-	Jackson est un analyseur (parser) de JSON. Il permet de décomposer une chaîne JSON pour en extraire des données et les transformer en objets Java. (ici : données passées depuis le service).

Utilitaire	Lombok	-	Réduit le code répétitif (boilerplate) comme les getters, setters, et constructeurs dans les classes Java grâce à des annotations (<i>ici</i> : utilisé dans les modèles).
Chiffrement	BCrypt	-	Hashage sécurisé des mots de passe.
Chiffrement	JASypt	3.0.5	Chiffrement et déchiffrement des informations sensibles dans les fichiers de configuration (<i>ici</i> : application.properties).
Conteneurisation	Docker	-	Conteneurisation de la base de données.
Orchestration	Docker Compose	-	Gestion des conteneurs multi-services.
Testing	Javafaker	1.0.2	Génération de données lambda en vue de tests.
Testing	H2 database	-	Base de données en-mémoire utilisée pour les tests.
Testing	Mockito	3.2.4	"Mocks" (objets factices) des dépendances (comme les services ou les repositories) pour isoler et tester une classe de manière unitaire. [<i>pas utilisé</i>]
Testing	JUnit	-	Framework de tests unitaires.

Frontend

Catégorie	Technologie	Version	Rôle
Framework	Angular	20.3.20	Framework principal pour le développement frontend.
Framework	Zone.js	0.15.x	Permet la détection de changement d'état (automatique); dépendance fondamentale d'Angular.
UI	Tailwind CSS	3.4.x	Framework CSS utilitaire pour la stylisation des composants.
UI	Angular Material	20.2.14	Framework Angular pour la stylisation des composants.
State Management	RxJS	7.8.x	Gestion réactive de l'état.
HTTP Client	Angular HttpClient	-	Client HTTP intégré à Angular.
Build	Angular CLI	20.3.20	Outil de compilation et gestion du projet.
API Client	OpenAPI Generator	2.31.x	Génération du client API.
Testing	Cypress	15.14.x	Tests composants (happy flows). (ici : pas de tests E2E configurés).
Testing	Karma - jasmine	-	Karma (le lanceur de tests) et Jasmine (le framework de test) ; combinaison par défaut d'Angular pour exécuter les tests unitaires. [<i>pas utilisé</i>]

Base de Données

Technologie	Version	Rôle
Microsoft SQL Server	2022	Système de Gestion de Base de Données principal pour le stockage des données.

Outils de Développement

Outil	Version	Rôle
IDE	IntelliJ IDEA	Environnement de développement intégré.
Build	Maven	Outil de gestion de projet et de build pour le backend. Il gère les dépendances (bibliothèques externes) via le fichier pom.xml et automatise le cycle de vie du projet.
Build	NPM	Gestionnaire de paquets pour le frontend. Il gère les dépendances (comme Angular, Tailwind) listées dans le fichier package.json et permet d'exécuter des scripts.
Versioning	Git	Système de contrôle de version.
CI/CD	GitHub Actions	Gestion de l'automatisation des workflows d'intégration et de déploiement continu.

Documentation de l'API

L'API backend est documentée via **Swagger UI** et est accessible lorsque l'application est démarrée localement.

URL Swagger

<http://localhost:8080/swagger-ui/index.html#/>

 **Note** : Cette URL est valable uniquement lorsque le serveur backend est en cours d'exécution. Aucune URL permanente n'est disponible. Le port utilisé est celui par défaut (défaut : 8080).

Fonctionnalités Swagger

- Visualisation interactive de tous les points d'entrée.
 - Test direct des requêtes via l'interface.
 - Documentation automatique des schémas de requêtes/réponses.
 - Authentification intégrée pour les points d'entrée sécurisés.
-

Conclusion

Ce dossier d'architecture présente une vue d'ensemble claire et structurée du projet *ProjetDV2026-PMALIIOUKOV*. L'architecture en couches permet une séparation nette des responsabilités, facilitant ainsi la maintenance, l'évolution et la collaboration.

Pour toute question ou clarification supplémentaire, veuillez consulter le code source ou contacter l'auteur du projet.

Auteur : Paul Malioukov (PSR10219)

Date : 29 mai 2026

Version : 2.0